# TECHNICAL ARCHITECTURE BLUEPRINT & ENGINEERING SPECIFICATION

***System Target:*** *Core Inventory & Parallel Demand Requisition Module Framework*
***Document Version:*** *6.0-Blueprint-Master*
***Framework Baseline:*** *Laravel Native v10+ / Eloquent ORM / MySQL 8.0+ / SHA-256 Chain Verification*
***Audience:*** *Backend Developers, Database Administrators, System Integration Engineers*

---

This engineering specification document accompanies the v6.0 Unified Master Systems Requirements Document (SRD). It bridges core warehouse stock movements, transactional point-of-care verification handshakes, parallel non-inventory intent collection streams, independent expiry quarantine states, and a cryptographically chained un-alterable forensic audit trail into direct database DDL blueprints and native Laravel service classes.

## 1. Complete Relational Database Schema Blueprint (DDL)

The schema migration layouts below organize relational indices, unique parameters, constraint actions, and tracking keys required across multi-site instances. Execution configuration matrices enforce an absolute isolation boundary separating physical warehouse stock data files from financial ledger blocks.

```
-- 1. TENANT CAPABILITY & EXPLICIT EXECUTION LAYERS
CREATE TABLE `tenant_capability_settings` (
    `id` BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    `tenant_id` VARCHAR(64) NOT NULL,
    `capability_key` VARCHAR(64) NOT NULL, -- e.g., 'FLOOR_STOCK', 'CRASH_CART', 'BATCH_TRACKING'
    `is_enabled` BOOLEAN DEFAULT FALSE NOT NULL,
    `created_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    `updated_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    UNIQUE KEY `uid_tenant_capability` (`tenant_id`, `capability_key`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- 2. DYNAMIC WAREHOUSE NODES & STRATEGY MATRICES
CREATE TABLE `inventory_stores` (
    `id` BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    `parent_id` BIGINT UNSIGNED NULL,
    `store_type` ENUM('MAIN', 'DISTRIBUTION', 'END_STORE', 'SATELLITE') NOT NULL,
    `crash_cart_status` ENUM('READY', 'DEPLOYED', 'RECONCILING') NULL DEFAULT NULL,
    `reorder_level_days` INT UNSIGNED DEFAULT 5 NOT NULL,
    `max_stock_level_days` INT UNSIGNED DEFAULT 20 NOT NULL,
    `created_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (`parent_id`) REFERENCES `inventory_stores`(`id`) ON DELETE RESTRICT,
    INDEX `idx_store_hierarchy` (`store_type`, `parent_id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE `store_stock_ledgers` (
    `id` BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    `store_id` BIGINT UNSIGNED NOT NULL,
    `inventory_sku` VARCHAR(64) NOT NULL,
    `physical_balance` INT NOT NULL DEFAULT 0,
    `location_layer_3` VARCHAR(64) NULL, -- Macro (e.g., Aisle, Wall, Tier)
    `location_layer_2` VARCHAR(64) NULL, -- Meso  (e.g., Rack, Cabinet, Tray)
    `location_layer_1` VARCHAR(64) NULL, -- Micro (e.g., Pallet, Bin, Slot)
    `updated_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    UNIQUE KEY `uid_store_sku` (`store_id`, `inventory_sku`),
    FOREIGN KEY (`store_id`) REFERENCES `inventory_stores`(`id`) ON DELETE RESTRICT,
    INDEX `idx_spatial_sorting` (`store_id`, `location_layer_3`, `location_layer_2`,
`location_layer_1`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- 3. RECONCILIATION REGISTERS & POINT-OF-CARE USAGE LOGGING
CREATE TABLE `patient_approved_pools` (
    `id` BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    `client_id` VARCHAR(64) NOT NULL,
    `inventory_sku` VARCHAR(64) NOT NULL,
    `approved_quantity` INT UNSIGNED DEFAULT 0 NOT NULL,
    `updated_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    UNIQUE KEY `uid_client_sku_pool` (`client_id`, `inventory_sku`),
    INDEX `idx_pool_lookup` (`client_id`, `inventory_sku`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE `point_of_care_usage_records` (
    `id` BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    `usage_context` ENUM('PATIENT', 'ADMINISTRATIVE', 'CRASH_CART', 'WASTAGE_OPERATIONAL',
'WASTAGE_EXPIRED') NOT NULL,
    `originating_store_id` BIGINT UNSIGNED NOT NULL,
    `client_id` VARCHAR(64) NULL,
    `administrative_purpose` VARCHAR(64) NULL,
    `inventory_sku` VARCHAR(64) NOT NULL,
    `quantity_consumed` INT UNSIGNED NOT NULL,
```

```
    `created_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (`originating_store_id`) REFERENCES `inventory_stores`(`id`) ON DELETE RESTRICT,
    INDEX `idx_usage_reconciliation` (`usage_context`, `client_id`, `inventory_sku`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- 4. HARDCODED PARALLEL DEMAND DATA TRACKING STRUCTURES
CREATE TABLE `inventory_demand_ledgers` (
    `id` BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    `client_id` VARCHAR(64) NOT NULL,
    `originating_client_space_id` VARCHAR(64) NOT NULL,
    `inventory_sku` VARCHAR(64) NOT NULL,
    `quantity_requested` INT UNSIGNED NOT NULL,
    `created_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    INDEX `idx_core_demand_engine` (`inventory_sku`, `created_at`),
    INDEX `idx_space_demand_analytics` (`originating_client_space_id`, `inventory_sku`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE `daily_consumption_snapshots` (
    `id` BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    `store_id` BIGINT UNSIGNED NOT NULL,
    `inventory_sku` VARCHAR(64) NOT NULL,
    `log_date` DATE NOT NULL,
    `total_units_consumed` BIGINT UNSIGNED DEFAULT 0 NOT NULL,
    UNIQUE KEY `uid_snapshot_timeline` (`store_id`, `inventory_sku`, `log_date`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE `daily_demand_snapshots` (
    `id` BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    `inventory_sku` VARCHAR(64) NOT NULL,
    `log_date` DATE NOT NULL,
    `total_units_demanded` BIGINT UNSIGNED DEFAULT 0 NOT NULL,
    UNIQUE KEY `uid_demand_timeline` (`inventory_sku`, `log_date`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- 5. HARDCODED FORENSIC CRYPTOGRAPHIC AUDIT LEDGER
CREATE TABLE `inventory_audit_ledgers` (
    `id` BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    `user_id` BIGINT UNSIGNED NOT NULL,
    `operational_context` VARCHAR(64) NOT NULL, -- e.g., 'DISPENSE', 'WASTAGE_EXPIRED'
    `store_id` BIGINT UNSIGNED NOT NULL,
    `client_id` VARCHAR(64) NULL,
    `inventory_sku` VARCHAR(64) NOT NULL,
    `old_quantity` INT NOT NULL,
    `new_quantity` INT NOT NULL,
    `node_hash` CHAR(64) NOT NULL, -- SHA-256 Block Chain Hash
    `created_at` TIMESTAMP(6) DEFAULT CURRENT_TIMESTAMP(6),
    INDEX `idx_forensic_audit` (`store_id`, `inventory_sku`, `created_at`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

## 2. Core Laravel Service Class Implementations

### 2.1 Point-of-Care Usage Recording & Reconciliation Engine

This runtime architecture manages the multi-scenario validation logic governed by **The Floor Principle**. It tracks physical drops, intercepts `WASTAGE_EXPIRED` actions to isolate stock into designated quarantine escrow balances, and dispatches asynchronous postpaid data arrays upstream to billing structures without changing local metrics.

```php
namespace App\Services\Inventory;

use Illuminate\Support\Facades\DB;
use Illuminate\Support\Facades\Http;
use App\Services\Inventory\AuditTrailEngine;
use Exception;

class UsageReconciliationEngine
{
    protected $auditTrail;

    public function __construct(AuditTrailEngine $auditTrail)
    {
        $this->auditTrail = $auditTrail;
    }

    /**
     * Entry-point for saving a clinical usage entry row.
     * Evaluates Scenario A, B, C, and Wastage splits atomically within a single transaction.
     */
    public function recordUsage(array $payload): array
    {
        return DB::transaction(function () use ($payload) {
            $context = $payload['usage_context'];
            $storeId = $payload['originating_store_id'];
            $sku = $payload['inventory_sku'];
            $qty = (int)$payload['quantity_consumed'];
            $userId = $payload['user_id'];
            $clientId = $payload['client_id'] ?? null;

            // Commit raw record line instantly to data files
            DB::table('point_of_care_usage_records')->insert([
                'usage_context' => $context,
                'originating_store_id' => $storeId,
                'client_id' => $clientId,
                'administrative_purpose' => $payload['administrative_purpose'] ?? null,
                'inventory_sku' => $sku,
                'quantity_consumed' => $qty,
                'created_at' => now()
            ]);

            // Query active physical stock metadata balances
            $currentStock = DB::table('store_stock_ledgers')
                ->where('store_id', $storeId)
                ->where('inventory_sku', $sku)
                ->lockForUpdate()
                ->first();

            $oldQty = $currentStock ? $currentStock->physical_balance : 0;

            switch ($context) {
                case 'PATIENT':
                    return $this->handlePatientUsage($userId, $storeId, $sku, $qty, $clientId,
$oldQty);

                case 'ADMINISTRATIVE':
                    $newQty = $oldQty - $qty;
                    $this->decrementStockAndAudit($userId, 'ADMIN_OPEX', $storeId, $sku, $qty,
$oldQty, $newQty, $clientId);
```

```
                    return ['scenario' => 'C', 'stock_reduced' => true, 'billing_triggered' => false];

                case 'CRASH_CART':
                    $newQty = $oldQty - $qty;
                    $this->decrementStockAndAudit($userId, 'CRASH_CART_DEDUCTION', $storeId, $sku,
$qty, $oldQty, $newQty, $clientId);
                    $this->dispatchBillingWebhook($clientId, $sku, $qty);
                    return ['scenario' => 'CRASH_CART', 'stock_reduced' => true, 'billing_triggered'
=> true];

                case 'WASTAGE_OPERATIONAL':
                    $newQty = $oldQty - $qty;
                    $this->decrementStockAndAudit($userId, 'WASTAGE_OPERATIONAL', $storeId, $sku,
$qty, $oldQty, $newQty, $clientId);
                    return ['scenario' => 'WASTAGE_OPERATIONAL', 'stock_reduced' => true,
'billing_triggered' => false];

                case 'WASTAGE_EXPIRED':
                    // EXPIRY QUARANTINE PROTOCOL: Drop stock, isolate items from moving averages via
snapshots
                    $newQty = $oldQty - $qty;
                    $this->decrementStockAndAudit($userId, 'WASTAGE_EXPIRED', $storeId, $sku, $qty,
$oldQty, $newQty, $clientId);
                    $this->routeToQuarantineEscrowZone($storeId, $sku, $qty);
                    return ['scenario' => 'WASTAGE_EXPIRED', 'stock_reduced' => true,
'billing_triggered' => false];

                default:
                    throw new Exception("Invalid usage context routing specified.");
            }
        });
    }

    private function handlePatientUsage($userId, $storeId, $sku, $qty, $clientId, $oldQty): array
    {
        $approvedPool = DB::table('patient_approved_pools')
            ->where('client_id', $clientId)
            ->where('inventory_sku', $sku)
            ->lockForUpdate()
            ->first();

        // SCENARIO A: Approved Item Usage (Prepaid Channel Handoff)
        if ($approvedPool && $approvedPool->approved_quantity >= $qty) {
            DB::table('patient_approved_pools')
                ->where('id', $approvedPool->id)
                ->decrement('approved_quantity', $qty);

            $this->auditTrail->logEvent($userId, 'PATIENT_APPROVED_POOL_USE', $storeId, $sku, $oldQty,
$oldQty, $clientId);
            return ['scenario' => 'A', 'stock_reduced' => false, 'billing_triggered' => false];
        }

        // SCENARIO B: Non-Approved Patient Usage (Postpaid Channel Handoff)
        $newQty = $oldQty - $qty;
        $this->decrementStockAndAudit($userId, 'NON_APPROVED_FLOOR_USE', $storeId, $sku, $qty,
$oldQty, $newQty, $clientId);
        $this->dispatchBillingWebhook($clientId, $sku, $qty);

        return ['scenario' => 'B', 'stock_reduced' => true, 'billing_triggered' => true];
    }
```

```
    private function decrementStockAndAudit($userId, $ctx, $storeId, $sku, $qty, $old, $new, $client):
void
    {
        DB::table('store_stock_ledgers')
            ->where('store_id', $storeId)
            ->where('inventory_sku', $sku)
            ->update(['physical_balance' => $new]);

        $this->auditTrail->logEvent($userId, $ctx, $storeId, $sku, $old, $new, $client);
    }

    private function dispatchBillingWebhook($clientId, $sku, $qty): void
    {
        $response = Http::timeout(3)->post(config('services.main_module.url') . '/api/v1/billing/
postpaid-event', [
            'client_id' => $clientId,
            'inventory_sku' => $sku,
            'quantity' => $qty,
            'event_type' => 'UNAPPROVED_CONSUMPTION_FACT'
        ]);

        if ($response->failed()) {
            DB::table('offline_event_journal')->insert([
                'payload_data' => json_encode(['client_id' => $clientId, 'sku' => $sku, 'qty' => $qty,
'type' => 'BILLING']),
                'created_at' => now()
            ]);
        }
    }

    private function routeToQuarantineEscrowZone($storeId, $sku, $qty): void
    {
        DB::table('quarantine_escrow_stocks')->updateOrInsert(
            ['store_id' => $storeId, 'inventory_sku' => $sku],
            ['quarantined_balance' => DB::raw("quarantined_balance + {$qty}"), 'updated_at' => now()]
        );
    }
}
```

### 2.2 Moving-Average Replenishment & Hardcoded Dual-Stream Engine

The updated calculation service handles dynamic toggles for parallel datasets on requisition order interfaces. Switching forecasting parameters alters mathematical indices seamlessly without modifying core libraries.

```
namespace App\Services\Inventory;

use Illuminate\Support\Facades\DB;
use Exception;

class MovingAverageReplenishmentEngine
{
    protected string $forecastingMode = 'consumption'; // Framework fallback baseline

    public function setForecastingMode(string $mode): self
    {
        if (!in_array($mode, ['consumption', 'demand'])) {
            throw new Exception("Invalid forecasting parameter target.");
        }
        $this->forecastingMode = $mode;
        return $this;
    }

    public function generateInternalOrderDraft(int $childStoreId): array
    {
        $storeConfig = DB::table('inventory_stores')->where('id', $childStoreId)->first();
        if (!$storeConfig) throw new Exception("Target store entry missing.");

        $targetCoverageDays = (int)$storeConfig->max_stock_level_days;
        $horizonWindow = $this->determineForecastingHorizon($targetCoverageDays);
        $lookbackDate = now()->subDays($horizonWindow)->toDateString();

        $activeItems = DB::table('store_stock_ledgers')->where('store_id', $childStoreId)->get();
        $compiledDraftLines = [];

        foreach ($activeItems as $item) {
            // DYNAMIC SWITCH ENGINE: Toggle mathematical velocity logic based on UI input parameter
            if ($this->forecastingMode === 'demand') {
                $dailyVelocity = DB::table('daily_demand_snapshots')
                    ->where('inventory_sku', $item->inventory_sku)
                    ->where('log_date', '>=', $lookbackDate)
                    ->avg('total_units_demanded') ?? 0.0;
            } else {
                $dailyVelocity = DB::table('daily_consumption_snapshots')
                    ->where('store_id', $childStoreId)
                    ->where('inventory_sku', $item->inventory_sku)
                    ->where('log_date', '>=', $lookbackDate)
                    ->avg('total_units_consumed') ?? 0.0;
            }

            // Standardize Current Stock Days along the strict 15-day baseline
            $currentStockDays = $this->calculateCurrentStockDays($childStoreId, $item->inventory_sku,
$item->physical_balance);

            if ($currentStockDays <= (int)$storeConfig->reorder_level_days) {
                $deficitDays = max(0, $targetCoverageDays - $currentStockDays);
                $suggestedQuantity = ceil($deficitDays * $dailyVelocity);

                if ($suggestedQuantity > 0) {
                    $compiledDraftLines[] = [
                        'inventory_sku' => $item->inventory_sku,
                        'current_balance' => $item->physical_balance,
                        'current_stock_days' => $currentStockDays,
                        'forecasting_basis' => strtoupper($this->forecastingMode),
```

```
                        'forecast_daily_velocity' => $dailyVelocity,
                        'suggested_quantity' => (int)$suggestedQuantity
                    ];
                }
            }
        }

        return [
            'requesting_store_id' => $childStoreId,
            'forecasting_mode_applied' => $this->forecastingMode,
            'horizon_window_days' => $horizonWindow,
            'line_items' => $compiledDraftLines
        ];
    }

    public function calculateCurrentStockDays(int $storeId, string $sku, int $currentQuantity): float
    {
        $fifteenDaysAgo = now()->subDays(15)->toDateString();
        $averageConsumption = DB::table('daily_consumption_snapshots')
            ->where('store_id', $storeId)
            ->where('inventory_sku', $sku)
            ->where('log_date', '>=', $fifteenDaysAgo)
            ->avg('total_units_consumed');

        if (!$averageConsumption || (float)$averageConsumption === 0.0) return 999.0;
        return (float)($currentQuantity / $averageConsumption);
    }

    private function determineForecastingHorizon(int $requestedCoverageDays): int
    {
        if ($requestedCoverageDays < 15) return 15;
        if ($requestedCoverageDays <= 30) return 30;
        if ($requestedCoverageDays <= 90) return 90;
        if ($requestedCoverageDays <= 180) return 180;
        return 365;
    }
}
```

### 2.3 Cryptographically Chained Forensic Audit Trail Engine

This hardcoded security engine records data deltas sequentially inside atomic routines. Row items calculate a verification hash bound directly to the digital signature string of the record immediately preceding it, building an immutable log tree.

```
namespace App\Services\Inventory;

use Illuminate\Support\Facades\DB;

class AuditTrailEngine
{
    /**
     * Commits a forensic point-in-time adjustment row with a sequential block verification hash.
     */
    public function logEvent(mixed $userId, string $ctx, int $storeId, string $sku, int $old, int
$new, ?string $client): void
    {
        // Fetch the absolute most recent cryptographic node signature from the log stream
        $lastLog = DB::table('inventory_audit_ledgers')
            ->orderBy('id', 'DESC')
            ->first();

        $previousHash = $lastLog ? $lastLog->node_hash : str_repeat('0', 64);
        $highPrecisionTimestamp = microtime(true);

        // Formulate clear plaintext verification chain string token
        $signaturePayload = implode('|', [
            $previousHash,
            $userId,
            $ctx,
            $storeId,
            $sku,
            $old,
            $new,
            $client ?? 'NULL',
            $highPrecisionTimestamp
        ]);

        $nodeHash = hash('sha256', $signaturePayload);

        DB::table('inventory_audit_ledgers')->insert([
            'user_id' => $userId,
            'operational_context' => $ctx,
            'store_id' => $storeId,
            'client_id' => $client,
            'inventory_sku' => $sku,
            'old_quantity' => $old,
            'new_quantity' => $new,
            'node_hash' => $nodeHash,
            'created_at' => now(6)
        ]);
    }
}
```

## 3. Multi-Service Core Communication API Contracts

### 3.1 Parallel Intent Ingestion Payload Specification

Dispatched down the wire via the Translator Engine directly to the Inventory Module ingestion endpoints to ensure data streams remain independent from forward payment clearance protocols.

```
HTTP PROTOCOL CONTEXT: POST | /api/v1/inventory/demand/capture-intent
JSON SPECIFICATION TARGET STRUCTURE:
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "ClinicalDemandIntentPayload",
  "type": "object",
  "required": ["client_id", "originating_space_id", "inventory_sku", "quantity_requested",
"timestamp"],
  "properties": {
    "client_id": { "type": "string", "maxLength": 64 },
    "originating_space_id": { "type": "string", "maxLength": 64 },
    "inventory_sku": { "type": "string", "maxLength": 64 },
    "quantity_requested": { "type": "integer", "minimum": 1 },
    "timestamp": { "type": "string", "format": "date-time" }
  },
  "additionalProperties": false
}
```

### 3.2 Counter Verification Token Handoff Callback Specification

Fires from End Store terminals straight to Clinical Module validation parameters to release staged totes without duplicate entries or local screen pollution.

```
HTTP PROTOCOL CONTEXT: POST | /api/v1/clinical/handshake/validate-token
JSON SPECIFICATION TARGET STRUCTURE:
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "TokenHandshakeCallback",
  "type": "object",
  "required": ["tote_identifier", "nurse_session_key", "verification_token"],
  "properties": {
    "tote_identifier": { "type": "integer" },
    "nurse_session_key": { "type": "string" },
    "verification_token": { "type": "string", "pattern": "^[0-9]{5}$" }
  },
  "additionalProperties": false
}
```

## 4. Operational State Change & System Flow Logic Matrices

### 4.1 Emergency Crash Cart Lifecycle Logic

The state machine transitions automatically along explicit events, locking clinical user entry blocks during resuscitations and postponing database item updates until the client session stabilizes.

| Initial State | Triggering Event Action | Target State | Programmatic Database Changes | Module Integration Effect |
|---|---|---|---|---|
| **`READY`** | User triggers `Deploy Crash Cart` or `Cart Opened` entry fields. | **`DEPLOYED`** | Updates row parameters inside the core `inventory_stores` configuration register. | Locks client screens; suppresses physical item tracking checks instantly during the crisis. |
| **`DEPLOYED`** | Staff clicks `Record Usage` for missing drugs after stabilization. | **`RECONCILING`** | Commits item logs to `point_of_care_usage_records` data tables. | Decrements local stock ledgers; dispatches data summary upstream to Main Billing gates. |
| **`RECONCILING`** | Technician completes physical restock and enters container seal number. | **`READY`** | Verifies full asset balances match structural requirements; wipes deficiency flags. | Generates forensic safety audit ledger signatures; re-locks physical cart container nodes. |

### 4.2 Multi-Mode Requisition Form Ordering Matrix

Altering ordering configurations modifies mathematical moving average properties dynamically, switching target source tables across parallel datasets inside your core codebase.

| Selected Mode Switch | Data Source Reference | Mathematical Logic Applied | Operational Procurement Purpose |
|---|---|---|---|
| **`CONSUMPTION`** | `daily_consumption_snapshots` table registry. | Calculates moving average velocities using actual physical drops from the shelves. | Maintains daily operational safety horizons; orders matching historical usage trends. |
| **`DEMAND`** | `daily_demand_snapshots` table registry. | Bases moving averages entirely on raw clinical intent requests, completely blind to stock counts. | Exposes unfulfilled deficits caused by supplier bottlenecks; calculates volumes to eliminate backlogs. |